

Creator Content Intelligence — Implementation Plan (v1)

Derived from the v1 spec (rev 6, June 2026). One sentence: turn a creator's entire content history into a search engine and assistant for fans — and turn fans' questions into a content plan and revenue for the creator.

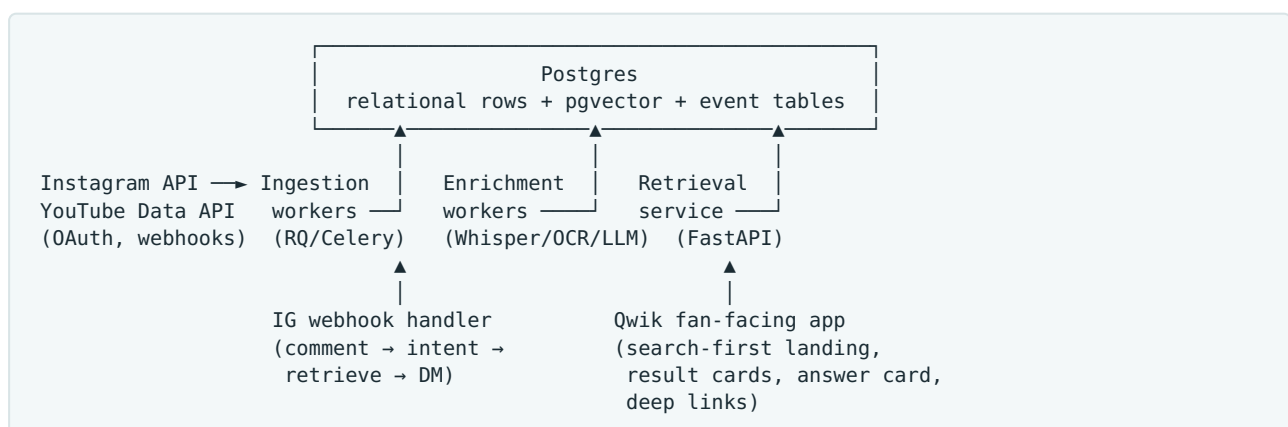
Guiding principle: the product is one loop — question → grounded cited answer → deep link → logged demand signal → creator insight (Demand Radar) → new content → better corpus → better answers. Search and the answer card are interfaces; **Demand Radar is the monetizable asset**. Retrieval quality is the hard part, not the chatbot.

1. Scope (v1 — ruthless)

In scope - The demand loop end to end: search-first archive + grounded answer card + Demand Radar. - One owned Instagram Business/Creator account (Standard Access), every record scoped to a creator. - Comments ingested from day one as demand signal. - Evaluation harness (40-60 labelled questions) from week one; citation correctness gates DM automation. - Comment-to-DM with **approval mode first**. - Deletion / reindexing support. - Light monetization: product mentions → creator-approved affiliate links.

Out of scope (defer) - Open-ended conversational chat (answer card only), voice cloning, mobile apps. - Full multi-tenant billing / self-serve onboarding (manual first). - Per-creator custom domains (path-based `yourapp.com/<creator>` first). - Dedicated vector DB (pgvector is plenty), broad analytics dashboards.

2. System architecture



Monorepo layout

```

octo/
├── apps/
│   ├── api/           # FastAPI: search, answer card, deep links, admin, webhooks
│   ├── web/           # Qwik City: fan-facing search-first pages ({creator})
│   └── workers/       # RQ/Celery: ingest, transcribe, OCR, enrich, embed, cluster
├── packages/
│   ├── core/          # domain models, DB access, creator scoping
│   ├── providers/     # thin interfaces: LLM, embeddings, transcription, OCR
│   └── retrieval/     # hybrid search, reranking, citation assembly
├── eval/              # labelled question set + scoring harness
├── infra/             # docker-compose, migrations, deploy
└── docs/

```

Stack (opinionated defaults, all swappable)

Layer	Choice
Ingestion	Instagram API with Instagram Login (owned account, no FB Page) + YouTube Data API (owned/authorized)
Transcription / OCR	faster-whisper + PaddleOCR behind one provider interface
Enrichment + answer card	Frontier LLM behind a thin provider interface (LiteLLM-style, swap by config)
Embeddings	One versioned model at a time; model + version + dims stored per chunk
Storage + vectors	PostgreSQL + pgvector (single DB)
Search	Hybrid: Postgres full-text + pgvector cosine → rerank
Events	PostHog (self-hostable) or plain Postgres event tables
Comment/DM loop	Instagram Messaging API + webhook handler
Backend / jobs	Python (FastAPI) + RQ/Celery; Rust (Axum) later for hot paths if needed
Frontend	Qwik City — near-zero JS first load for Instagram's in-app browser
Hosting	Docker containers anywhere; S3-compatible object storage for media

Portability principles (non-negotiable) - Every AI call (LLM, embeddings, transcription, OCR) goes through a provider interface; vendor swap = configuration. - Embedding swaps are planned re-embed jobs, never a mixed index. - Plain Postgres + pgvector + S3-compatible storage; no silent lock-in on managed services.

3. Data model (core tables)

creators	(id, handle, display_name, ig_user_id, yt_channel_id, status, ...)
oauth_tokens	(creator_id, platform, access_token, refresh_token, expires_at, scopes)
posts	(id, creator_id, platform, external_id, type, caption, permalink, posted_at, media_url, status[active deleted], ingested_at)
media_assets	(post_id, kind[video image audio], storage_key, duration, ...)
transcripts	(post_id, source[whisper yt_caption], text, segments JSONB, quality)
ocr_texts	(post_id, frame_ts, text, confidence)
enrichments	(post_id, summary, topics[], keywords[], product_mentions JSONB, location_mentions JSONB, model, version)
chunks	(id, post_id, creator_id, source[caption transcript ocr summary], text, embedding vector, embed_model, embed_version, embed_dims, tsvector, created_at) -- design for delete + reindex
comments	(id, creator_id, post_id, external_id, author_pseudonym, text, created_at, is_question bool, intent, sync_watermark)
queries	(id, creator_id, source[search dm comment], text, normalized_text, result_post_ids[], answer_id, confidence, created_at) -- pseudonymous
answers	(id, query_id, text, citations JSONB, confidence, state[answered no_strong_answer], model, created_at)
dm_jobs	(id, comment_id, status[pending_approval approved sent fallback failed], public_reply, dm_text, deep_link, sent_at)
demand_topics	(id, creator_id, label, week, search_count, comment_count, wow_change, coverage JSONB, audience_language JSONB, recommendation, linked_products JSONB, confidence, creator_marked[useful not made])
products	(id, creator_id, name, affiliate_url, approved bool)
product_links	(post_id ↔ product_id, source[enrichment manual])
events	(id, creator_id, kind[search result_click deeplink_open dm_sent affiliate_click share], payload JSONB, ts) -- powers all \$8 metrics
eval_questions	(id, creator_id, question, expected_post_ids[], labelled_by, ...)
eval_runs	(id, git_sha, config_hash, top3_accuracy, citation_correctness, no_answer_accuracy, created_at)

Key constraints: - **Everything is creator-scoped** — `creator_id` on every row, enforced in the data-access layer from day one (multi-tenant-ready even with one creator). - **Deletion-first design**: deleting a post cascades to chunks/citations; nightly sync marks deleted posts; stale citations kill trust. - **Pseudonymous audience data** (GDPR): hash commenter/searcher identifiers; store aggregates for Radar; deletion path before public launch.

4. Pipelines

4.1 Ingestion

1. OAuth connect (Instagram Login, scopes: `instagram_business_basic`, `_manage_comments`, `_manage_messages`) → store tokens, create creator namespace.
2. Backfill posts: paginate media endpoints; download media to object storage.
3. Backfill comments: cursor-walk (~50/page, no timestamp filter) → keep our own **watermark** per post for incremental sync. Batch + cache (API budget per account).
4. Incremental sync: scheduled job + webhooks; detect deletions and edits.
5. **Never scrape**. Official APIs only.

4.2 Enrichment

1. Transcribe video/audio (faster-whisper) with segment timestamps.
2. OCR keyframes (PaddleOCR) for burned-in text; dedupe per post.
3. LLM enrichment per post: summary, topics, keywords, product/location mentions → structured JSON, creator-scoped.

4.3 Indexing

1. Chunk caption + transcript segments + OCR + summary (per-source chunkers, ~200–400 token targets, overlap on transcripts).
2. Embed with the single versioned model; store model/version/dims per chunk.
3. Maintain `tsvector` for full-text alongside.
4. Reindex job: triggered by post edit/delete, enrichment change, or embedding-model swap (full re-embed migration).

4.4 Retrieval & answer card

1. Hybrid retrieve: full-text + vector cosine in parallel → merge → rerank (LLM or cross-encoder) → top-k post-level results with chunk evidence.
2. Answer card: grounded generation **only from retrieved chunks**, citation per claim, bounded length, explicit **"no strong answer"** state below a confidence threshold.
3. Retrieval answer is computed separately from any creator-voice styling — correctness and citations first; voice is a later layer.
4. Every result and answer has a stable, shareable **deep link** (`/{{creator}}?q=...&a=...`).

4.5 Comment-to-DM (capture layer, approval mode first)

1. Webhook receives comment → intent detection (question vs. noise vs. trigger keyword).
2. Question → same retrieval layer → answer + deep link.
3. **Approval queue**: creator/operator approves each public-reply + DM pair until the citation gate passes and compliance is proven; then automate per-post with specific trigger CTAs ("Comment PLAN...").
4. Hard compliance bounds: one private reply per comment, within 7 days; follow-ups only if the person replies; respect ~200 DMs/hour cap with a queue; **low confidence → friendly archive link, never a shaky answer in the creator's voice**.

4.6 Demand Radar

1. Cold start: cluster the **historical comment backlog** so the creator sees value before any search traffic.
2. Weekly job: cluster queries + question-comments (embedding clustering + LLM labelling) → evidence cards: topic, signal counts + WoW change, current coverage (matched posts + gaps), audience language verbatims, recommended content, linked products, confidence.
3. Delivery: **weekly digest pushed via email/DM** (creators don't visit dashboards) + a simple review UI to mark cards useful / not / "made it".

5. Evaluation harness (week 1, runs forever)

- 40–60 real audience questions mined from the comment backlog + repeated DMs; ~2–3 h creator labelling to map each to expected post(s).
 - Scored on every change to OCR / chunking / embeddings / reranking:
 - **Top-3 retrieval accuracy** (gate: beats native IG search on the labelled set).
 - **Citation correctness** (gate: must pass on a held-out set before any DM automation goes live).
 - **"No answer" accuracy** on out-of-corpus questions.
 - Results persisted per git SHA + config hash (`eval_runs`); CI fails on regression.
-

6. Build sequence (~10 weeks)

Phase	Focus	Exit gate
0 • Unblock data (day 1)	Create Meta app; connect owned account via Instagram Login (Standard Access — no review). Submit App Review in parallel (only scopes the demo uses) for external creators. YouTube API key. Repo scaffold: docker-compose (Postgres+pgvector, MinIO, Redis), FastAPI skeleton, migrations, provider interfaces.	Media + comments pullable the same week; review in flight.
1 • Ingest + evaluate (wk 1–2)	Full ingestion pipeline (posts, media, comments from day one), transcribe, OCR, enrich, chunk, embed. Stand up the labelled question set in week 1.	100+ items indexed; eval harness running.
2 • Search-first link (wk 3–4)	<code>yourapp.com/<creator></code> : one search box (auto-focused, instant load in IG in-app browser, no login wall), cited result cards, deep links. Event tracking live. No homepage.	Beats native IG search on the labelled set; a follower finds a known post in a couple of taps.
3 • Demand Radar (wk 5–6)	Cluster comment backlog + searches into evidence cards (crude is fine — even a spreadsheet first); weekly digest delivery.	Creator marks which of 10 opportunities they'd actually make.
4 • Answer card (wk 7)	Grounded, cited answer card with "no strong answer" state — not open chat.	Citation correctness passes the gate on a held-out set.
5 • Comment-to-DM (wk 8–9)	Webhook + intent detection on a few posts with specific trigger CTAs; first DM carries the answer + deep link; approval mode first ; low-confidence fallback to archive link; rate-limit queue.	A real comment yields a compliant, value-first auto-DM.
6 • Monetization proof (wk 10)	Product mentions → creator-approved affiliate links on results and answers; click tracking.	≥1 live affiliate link; click data supports the paid pitch.

Paid pilot conversations start at Phase 3, as soon as Radar shows useful cards.

7. Distribution checklist (creator onboarding)

- Bio link titled clearly (IG allows up to 5 native links).
- Permanent **"Search" Highlight** (Story link sticker saved to a Highlight — stays clickable).
- Pinned reel pointing at the search page.
- Creator behaviour script: repeatedly tell the audience "my whole archive is searchable — tap Search on my profile."
- Comment-to-DM CTAs tested **post by post** ("Comment PLAN and I'll send the exact routine" > generic "ask me anything").

8. Metrics (events from day one)

Area	Metric
Ingestion	% posts with usable caption / transcript / OCR text
Search quality	Top-3 retrieval accuracy on labelled set
Answer quality	Citation correctness, unsupported-claim rate
Audience UX	Search→click rate, repeat searches, time-to-answer
Deep links	Share / open rate per result URL
Comment-to-DM	Trigger rate, DM send success, deep-link click rate
Demand Radar	% insights marked "useful", posts created from insights
Creator ROI	Affiliate/product clicks, support time saved
Safety	"No answer" accuracy, hallucination rate, stale-citation rate

The two metrics that decide it: (1) can a real follower find the right old post faster than through Instagram? (2) does the creator act on Demand Radar without being persuaded?

9. Risks → mitigations

Risk	Mitigation
API / App Review dependency	POC on Standard Access (owned account, no review); submit review day one; ingest owned YouTube in parallel; link-first product stays valuable if DM lags.
Hallucination / brand damage	Strict grounding, visible citations, citation gate before automation; "no answer" state.
DM compliance	Official API only; 1 value-first reply per comment ≤ 7 days; never unsolicited; honor rate caps.
Weak creator pull	Radar woven into the weekly content workflow; require Highlight + pinned reel.
Insufficient audience usage	Comments ingested day one → Radar has signal before search ramps.
Meta ships it natively	Own the cross-platform corpus, creator-side demand intelligence, affiliate layer.
Privacy / GDPR	Pseudonymous + aggregated queries; privacy policy + deletion path pre-launch.
Overbuilding the chatbot	Answer card on proven retrieval; no open chat in v1.

10. Pricing hypothesis (validate from Phase 3)

- **Free** — public audience search page with "Powered by" footer.
- **Creator ~\$49-99/mo** — Radar weekly digest, answer card, affiliate mapping, email capture (v1.5).
- **Pro ~\$149-299/mo** — comment-to-DM automation, storefront search, draft generator, sponsor reports, multi-platform ingestion.

Expansion ladder (v1.5/v2, each rides the existing retrieval layer): email capture bridge → storefront search → embeddable widget → insight-to-draft generator → sponsor pitch reports → paid Q&A on "no answer".